- *Write side*—The implementation of a handler to convert the data to the new storage (for example, an upsert) and a connection pool (such as a database session)

These are shown in the following listing.

Listing 10.5 Akka projection ingredients

```
import akka.projection.eventsourced.scaladsl.EventSourcedProvider
import akka.persistence.jdbc.query.scaladsl.JdbcReadJournal
import example.repository.scalike.ScalikeJdbcSession
import example.persistence.SPContainer

object CargosPerContainerProjection {

    ...

    val sourceProvider =
      EventSourcedProvider.eventsByTag[SPContainer.Event](        ❶
        system = system,
        readJournalPluginId = JdbcReadJournal.Identifier,
        tag = tag)

    JdbcProjection.exactlyOnce(
      projectionId =
        ProjectionId("CargosPerContainerProjection", tag),
      sourceProvider = sourceProvider,
      handler =
        () => new CPCProjectionHandler(tag, system, repository),  ❷
      sessionFactory = () => new ScalikeJdbcSession())            ❸
    (system)
}
```

❶ Reads from the event journal; read side

❷ Handler to upsert into the database; write side

❸ Pool of connections to provide to the handler; write side

Let's look at these three building blocks

10.3.1 Reading

To read from the database, you need the `EventSourcedProvider` specifying that it reads `eventsByTag` from the `SPContainer`. That is `SPContainer.Event`.

The projection runs on the `ActorSystem`. The `readJournalPluginId` selects the plugin to read the event. If the events were stored in Cassandra, you would select `CassandraReadJournal.Identifier` instead.

Finally, the tag must match the tags from the journal, so you need to be aware of the tags the application is generating—in this example, `container-tag-[0|1|2]`.

10.3.2 Writing

To write to the RDBMS, you need a handler that extends `JdbcHandler[EventEnvelope[SPContainer.Event], ScalikeJdbcSession]` and a repository to define the SQL statement. This handler must override the `process` method from the `JdbcHandler`